数学基础急救包

专门给"数学忘光了"的你 —— 学到哪，补到哪

先给你吃颗定心丸：机器学习用到的数学，**并不需要高深**。真正反复用到的，其实就下面这几个"零件"：变量、函数、平方、平均数、求和符号 Σ、直线方程。它们全都是**初中水平**，你只是忘了，不是学不会。

怎么用这个笔记本：

- 不用一次学完！把它当成你的**工具抽屉**，学到需要哪个零件，就回来拿哪个。
- 每个概念都配一个 **生活版解释**（先搞懂"它到底在干嘛"）+ **一小段能跑的代码**（亲眼看到它动起来）。
- 代码格点一下、按 `Shift + Enter` 就能运行。看不懂的先跳过，重要的是**感觉**。

> 记住我们的口号：**先理解"它在干嘛"，再管"符号怎么写"。** 数学符号只是"简写"，不是新的魔法。

第 0 步：环境自检

运行下面这格，确认画图工具就绪。看到"就绪"就继续。

In [1]:

```
try:
    import numpy as np
    import matplotlib.pyplot as plt
    print(" 环境就绪，可以继续啦！")
except ImportError:
    print(" 请先在命令行运行：pip install numpy matplotlib")
```

```
环境就绪，可以继续啦！
```

---

1. 字母代表数（变量）

数学里老爱用字母 `x`、`y`、`w`、`b` 来表示"某个数"。**它不是什么神秘的东西**，你就想象成**给一个盒子贴了标签**：盒子叫 `x`，里面装哪个数，要看具体情况。

比如我说"`x` = 房子的面积"。那 `x` 到底是 120 还是 80？取决于我们在说哪一套房子。**字母 x 是固定的名字，里面的数值可以变，所以叫"变量"。**

看代码：

In [2]:

```
x = 120          # 把数字 120 装进名叫 x 的盒子
print("x 现在是：", x)

x = 85           # 同一个盒子，换成装 85
print("x 现在变成：", x)
# 你看，x 这个名字没变，但里面装的数可以变 —— 这就是"变量"
```

```
x 现在是： 120
x 现在变成： 85
```

2. 什么是"函数"

函数 = 一台"输入 → 输出"的小机器。 你丢一个数进去，它按固定规则吐一个数出来。

生活类比：榨汁机

- 放进橙子 → 出来橙汁
- 放进苹果 → 出来苹果汁
- 只要"规则"（榨汁）不变，同样的输入永远得到同样的输出。

写成 `f(x)`：读作"**f 作用于 x**"。`f` 是这台机器的名字，`x` 是你丢进去的东西，`f(x)` 就是**吐出来的结果**。

举个具体例子：`f(x) = 2x + 1`，规则是"把进来的数乘以 2，再加 1"。

- 丢进 3 → 出来 2×3+1 = **7**
- 丢进 10 → 出来 2×10+1 = **21**

看代码：

In [3]:

```
# 定义一台名叫 f 的机器，规则是：2x + 1
def f(x):
    return 2 * x + 1      # 丢进来 x，吐出去 2x+1

print("f(3)  =", f(3))    # 丢进 3
print("f(10) =", f(10))   # 丢进 10
print("f(0)  =", f(0))    # 丢进 0

# 机器学习干的，就是"自己找出这台机器的规则"！
```

```
f(3)  = 7
f(10) = 21
f(0)  = 1
```

3. 平方 和 平均数

这两个是后面算"误差"时要反复用的。

平方：一个数自己乘自己。写成 3^2，读作"3 的平方"，等于 3×3 = **9**。

> 为什么老用平方？因为**负的平方会变成正的**。比如误差 -2 和 +2，直接相加会互相抵消变成 0（好像没误差一样）；一平方，就都变成 +4，不会被抵消。这个道理等你学到"成本函数"时，会感受得特别深。

平均数：把所有数加起来，再除以个数。

- 2、4、6 的平均数 = (2+4+6) ÷ 3 = **4**

看代码：

In [4]:

```
print("3 的平方 =", 3**2)         # ** 在 Python 里表示"平方"
print("(-2)的平方 =", (-2)**2)     # 负数的平方也是正的

nums = [2, 4, 6]
print("平均数 =", sum(nums) / len(nums))   # 加起来 除以 个数
# sum 是求和，len 是数一数有几个
```

```
3 的平方 = 9
(-2)的平方 = 4
平均数 = 4.0
```

---

4. 求和符号 Σ（像个横过来的大 E）

Σ 是希腊字母 **Sigma**，音译"西格玛"。它的全部意思就一句话：

> **"把后面这些东西，一个不落地全加起来。"**

看到没，它**不是什么高深的运算**，就是一个"加"字的高级写法。

写法：$\Sigma_{i=1}^{n} x_i$，读作"把 x_1 到 x_n 全加起来"。

- 底下 `i=1` 表示"从第 1 个开始"
- 顶上 `n` 表示"加到第 n 个"
- 举个小的：如果 x = [2, 5, 3]，那么 $\Sigma x_i = 2 + 5 + 3 = \mathbf{10}$

看代码——你会看到 Σ 在 Python 里就是 `sum()`：

In [5]:

```
x = [2, 5, 3]
print("Σxᵢ =", sum(x))      # 求和符号 Σ 干的活，就是 sum()

# 再多点数据体会一下
scores = [88, 92, 79, 95]
print("总分Σ 分数 =", sum(scores))
```

```
Σxᵢ = 10
总分Σ 分数 = 354
```

---

5. 直线方程 y = wx + b （最重要的一节）

这是线性回归的**主角**，一定要搞懂。先看它长什么样：

```
y = w * x + b
```

先别怕，我们逐个字母拆开。它其实就是在说："**先拿 x 乘以 w，再加上 b，就得到 y**"。就是一段乘法加法的算术。

关键在 w 和 b 这两个"旋钮"，它们控制这条直线长什么样：

- **w（斜率）= 控制"陡不陡"**。w 越大，线越陡（越"斜"）；w 越小，线越平。

- **b（截距）= 控制"上下平移"**。b 就是这条线和竖轴（y 轴）交在哪个高度；b 变大，整条线往上抬。

下面两张图，一张只看 w 变，一张只看 b 变，你一看就懂：

In [6]:

```
import numpy as np
from matplotlib import font_manager
# 让图里能显示中文
# ---- 自动挑一个你电脑上真实存在的中文字体 ----
# Windows 用微软雅黑/黑体，Mac 用苹方，Linux 用 Noto；谁在就用谁
_candidates = ["Microsoft YaHei", "SimHei", "PingFang SC",
               "Noto Sans CJK JP", "Noto Sans CJK SC", "Droid Sans Fallback"]
_available = {f.name for f in font_manager.fontManager.ttflist}
_chosen = [c for c in _candidates if c in _available] or ["DejaVu Sans"]
plt.rcParams["font.sans-serif"] = _chosen
plt.rcParams["axes.unicode_minus"] = False
print("本次使用中文字体：", _chosen)

x = np.linspace(-3, 3, 100)   # 一串从 -3 到 3 的 x 值

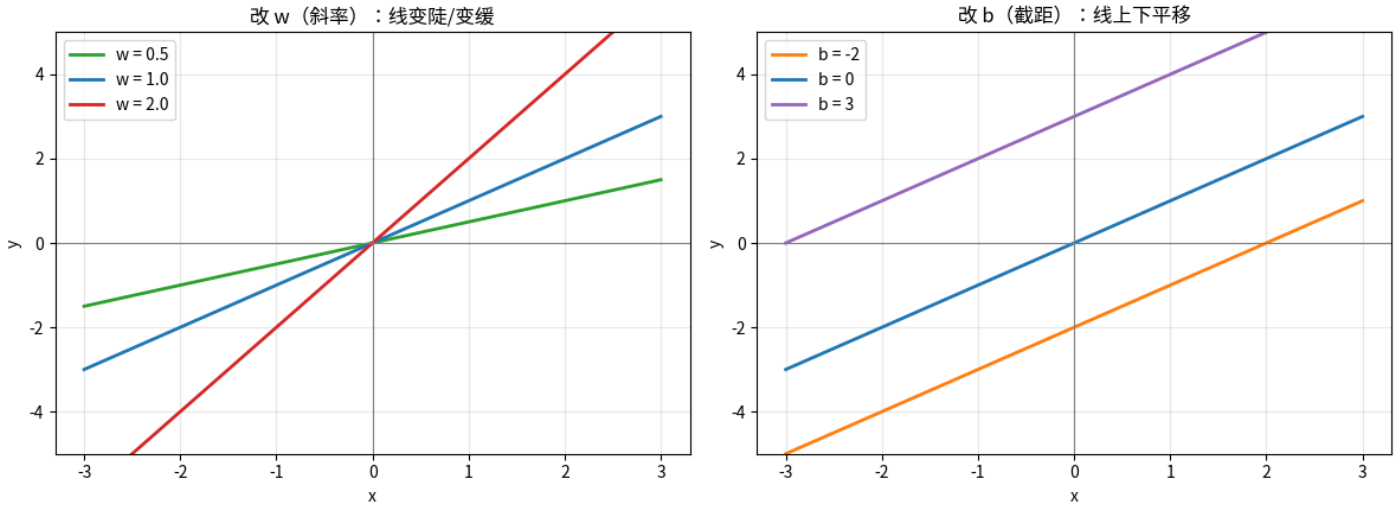
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4.5))

# 左图：固定 b=0，只改 w（斜率）
for w, col in [(0.5, "tab:green"), (1.0, "tab:blue"), (2.0, "tab:red")]:
    ax1.plot(x, w * x + 0, color=col, linewidth=2, label=f"w = {w}")
ax1.set_title("改 w（斜率）：线变陡/变缓")
ax1.set_xlabel("x"); ax1.set_ylabel("y")
ax1.axhline(0, color="gray", linewidth=0.8); ax1.axvline(0, color="gray", linewidth=0.8)
ax1.legend(); ax1.grid(alpha=0.3); ax1.set_ylim(-5, 5)

# 右图：固定 w=1，只改 b（截距）
for b, col in [(-2, "tab:orange"), (0, "tab:blue"), (3, "tab:purple")]:
    ax2.plot(x, 1 * x + b, color=col, linewidth=2, label=f"b = {b}")
ax2.set_title("改 b（截距）：线上下平移")
ax2.set_xlabel("x"); ax2.set_ylabel("y")
ax2.axhline(0, color="gray", linewidth=0.8); ax2.axvline(0, color="gray", linewidth=0.8)
ax2.legend(); ax2.grid(alpha=0.3); ax2.set_ylim(-5, 5)

plt.tight_layout(); plt.show()
```

```
本次使用中文字体： ['Noto Sans CJK JP', 'Droid Sans Fallback']
```



看明白了吗：

- 左图里三条线都从原点出发，但**越陡的线 w 越大**（红 w=2 最陡，绿 w=0.5 最平）。
- 右图里三条线**倾斜一样**（都 w=1），但**上下不同的位置**——高矮就是 b 决定的。

> 所以你以后一看到 `f(x) = wx + b`，脑子里要立刻翻译成："**一条可以调陡度(w)、可以调高低(b)的直线。**"
>
> 而**机器学习要做的，就是自动找出最合适的 w 和 b**，让这条线尽量贴着数据走。

---

6. 下标记号：x_1、x_2 和 $x^{(i)}$

下标用来给一堆同类的数编号，就像给排队的人发号码牌：

- x_1 = 第一个 x，x_2 = 第二个 x，x_3 = 第三个 x……
- x = [10, 20, 30]，那么 x_1=10、x_2=20、x_3=30

特别提醒（超容易搞混）：在机器学习里，你会看到 $x^{(i)}$ 这种**带括号的上标**，它表示"**第 i 个训练样本**"，**不是"x 的 i 次方"！** 括号就是用来跟你区分"次方"的。

看了代码就清楚了：

In [7]:

```
x = [10, 20, 30]
# 数学里写 x₁，代码里从 0 开始数，所以 x₁ 对应 x[0]
print("x₁(第1个) =", x[0])
print("x₂(第2个) =", x[1])
print("x₃(第3个) =", x[2])

# 上标 x⁽ⁱ⁾ 表示"第 i 个样本"，不是次方！比如第 2 个样本：
i = 2
print(f"第{i}个样本 x⁽²⁾ =", x[i-1])
```

```
x₁(第1个) = 10
x₂(第2个) = 20
x₃(第3个) = 30
第 2 个样本 x⁽²⁾ = 20
```

随时回来查：小抄

符号	读法	一句话意思
`x`	变量	一个盒子里的数
`f(x)`	f 作用于 x	把 x 丢进机器 f，得到输出
`x²`	x 的平方	x 乘自己
`Σ`	西格玛求和	全部加起来
`y = wx + b`	直线方程	w 控陡度，b 控高低
`xᵢ`	x 下标 i	第 i 个数
`x⁽ⁱ⁾`	x 上标 i（带括号）	第 i 个**样本**，不是次方！

补完这些，你就有足够"零件"去学第 2 课的公式了。 哪里没看懂，随时回来问我，我们把那一个零件单独拆开讲。